

Тема урока: условный оператор

В Python существует несколько способов проверки, и в каждом случае возможны два исхода: истина (`True`) или ложь (`False`).



Проверка условий и принятие решений по результатам этой проверки называется **ветвлением** (branching). Программа таким способом выбирает, по какой из возможных ветвей ей двигаться дальше.

В Python проверка условия осуществляется при помощи ключевого слова `if`.

Рассмотрим следующую программу:

```
answer = input('Какой язык программирования мы изучаем?')
if answer == 'Python':
    print('Верно! Мы ботаем Python =)')
    print('Python - отличный язык!')
```

Программа просит пользователя ввести текст и проверяет результат ввода. Если введенный текст равен строке «Python», то выводит пользователю текст:

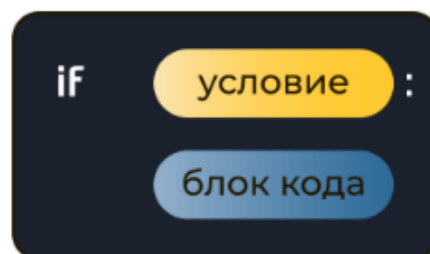
```
Верно! Мы ботаем Python =)
Python - отличный язык!
```

Двоеточие (`:`) в конце строки с инструкцией `if` сообщает интерпретатору Python, что дальше находится **блок команд**. В блок команд входят все строки с отступом под строкой с инструкцией `if`, вплоть до следующей строки без отступа.

Если условие истинно, выполняется весь расположенный ниже блок. В предыдущем примере блок инструкций составляет третья и четвертая строки программы.



Блоком кода называют объединенные друг с другом строки. Они всегда связаны с определенной частью программы (например, с инструкцией `if`). В Python блоки кода формируются при помощи **отступов**.

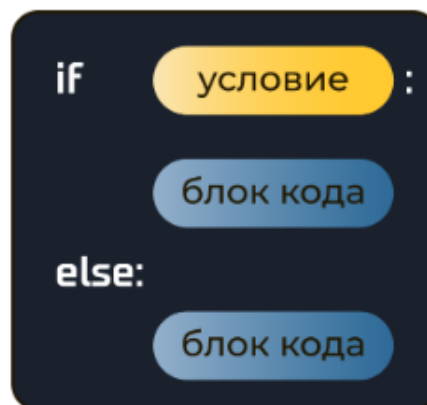


Предыдущая программа выводит текст в случае, если условие истинно. Но если условие ложно, то программа ничего не выводит. Для того чтобы обеспечить возможность выполнять что-либо в случае, если условие оказалось ложным, мы используем ключевое слово `else`.

```
answer = input('Какой язык программирования мы изучаем?')

if answer == 'Python':
    print('Верно! Мы ботаем Python =)')
    print('Python - отличный язык!')
else:
    print('Не совсем так!')
```

В новой программе мы обрабатываем сразу два случая: если условие истинно (пользователь ввел «Python»), и если условие ложно (пользователь ввел что угодно, кроме «Python»).



Отступы

В некоторых языках программирования отступы — дело личного вкуса, и можно вообще обходиться без них. Однако в Python они — неотъемлемая часть кода. Именно отступ сообщает интерпретатору Python, где начинается и где заканчивается блок кода.



Отступ — небольшое смещение строки кода вправо. В начале такой строки находятся пробелы, и поэтому она на несколько символов отстоит от левого края.

Некоторым инструкциям в Python (например, инструкции `if`) именно блок кода сообщает, какие действия следует предпринять. После `if` блок кода информирует интерпретатор Python, как действовать, если условие истинно, и как — если оно ложно.



По соглашению PEP 8, для отступа блоков кода используются **4 пробела**. Если в среде VS Code или Wing IDE нажать на клавишу Enter после `if`, она автоматически выставит **4 пробела**.

Операторы сравнения

Можно заметить, что в проверке условия мы использовали двойное равенство (`==`), вместо ожидаемого одиночного (`=`). Не стоит путать **оператор присваивания** (`=`) с **условным оператором** (`==`).

Оператор присваивания (`=`) присваивает переменным значения:

```
num = 1992
s = 'I love Python'
```

Для проверки двух элементов на равенство Python использует удвоенный знак равно (`==`). Вот так:

```
if answer == 'Python':
    ...

if name == 'Gvido':
    ...

if temperature == 40:
    ...
```



Путаница с операторами `==` и `=` является одной из самых распространенных ошибок в программировании. Эти символы используются не только в Python, и каждый день множество программистов используют их неправильно.

В Python существует 6 основных операторов сравнения.

Выражение	Описание
<code>if x > 7</code>	если x больше 7
<code>if x < 7</code>	если x меньше 7
<code>if x >= 7</code>	если x больше либо равен 7
<code>if x <= 7</code>	если x меньше либо равен 7
<code>if x == 7</code>	если x равен 7
<code>if x != 7</code>	если x не равен 7

Рассмотрим пример:

```
num1 = int(input())
num2 = int(input())

if num1 < num2:
    print(num1, 'меньше чем', num2)
if num1 > num2:
    print(num1, 'больше чем', num2)

if num1 == num2: # используем двойное равенство
    print(num1, 'равно', num2)
if num1 != num2:
    print(num1, 'не равно', num2)
```

Цепочки сравнений

Операторы сравнения в Python можно объединять в цепочки (в отличие от большинства других языков программирования, где для этого нужно использовать логические связки), например, `a == b == c` или `1 <= x <= 10`. Следующий код проверяет, находится ли значение переменной `age` в диапазоне от 3 до 6:

```
age = int(input())
if 3 <= age <= 6:
    print('Вы ребёнок')
```

Код, проверяющий равенство трех переменных, может выглядеть так:

```
if a == b == c:
    print('числа равны')
else:
    print('числа не равны')
```

Транзитивность

Операция равенства является **транзитивной**. Это означает, что если `a == b` и `b == c`, то из этого следует, что `a == c`. Именно поэтому предыдущий код, проверяющий равенство трех переменных, работает, как предполагается.

Из курса математики вам могут быть знакомы другие примеры транзитивных операций:

- **Отношение порядка:** если $a > b$ и $b > c$, то $a > c$;
- **Параллельность прямых:** если $a \parallel b$ и $b \parallel c$, то $a \parallel c$;
- **Делимость:** если a делится на b и b делится на c , то a делится на c .

Наглядно транзитивность отношения порядка можно понять на таком примере: если сосед слева старше вас ($a > b$), а вы старше соседа справа ($b > c$), то сосед слева точно старше соседа справа ($a > c$).

Операция неравенства (`!=`), в отличие от операции равенства (`==`), является **нетранзитивной**. То есть из того, что `a != b` и `b != c` вовсе не следует, что `a != c`. Действительно, если вас зовут не так, как соседа слева и не так, как соседа справа, то нет гарантии, что у обоих соседей не окажутся одинаковые имена.

Таким образом, следующий код вовсе **не проверяет** тот факт, что все три переменные различны:

```
if a != b != c:
    print('числа не равны')
else:
    print('числа равны')
```

Решение задач

Задача 1. Напишите программу, которая считывает одну строку. Если это строка «Python», программа выводит «ДА», в противном случае программа выводит «НЕТ».

Решение. Программа, решающая поставленную задачу, может иметь вид:

```
word = input()

if word == 'Python':
    print('ДА')
else:
    print('НЕТ')
```

Задача 2. Напишите программу, которая определяет, состоит ли двузначное число, введенное с клавиатуры, из одинаковых цифр. Если состоит, то программа выводит «ДА», в противном случае программа выводит «НЕТ».

Решение. Программа, решающая поставленную задачу, может иметь вид:

```
num = int(input())

last_digit = num % 10    # последняя цифра числа
first_digit = num // 10  # первая цифра числа

if last_digit == first_digit:
    print('ДА')
else:
    print('НЕТ')
```

Задача 3. Напишите программу, которая считывает три числа и подсчитывает количество чётных чисел.

Решение. Программа, решающая поставленную задачу, может иметь вид:

```
num1, num2, num3 = int(input()), int(input()), int(input())

counter = 0 # переменная счётчик
if num1 % 2 == 0:
    counter = counter + 1 # увеличиваем счётчик на 1
if num2 % 2 == 0:
    counter = counter + 1 # увеличиваем счётчик на 1
if num3 % 2 == 0:
    counter = counter + 1 # увеличиваем счётчик на 1

print(counter)
```

Тема урока: логические операторы

Логические операторы

Как быть в ситуации, когда у нас есть несколько условий? В Python есть три логических оператора, которые позволяют создавать сложные условия:

- `and` — логическое умножение;
- `or` — логическое сложение;
- `not` — логическое отрицание.

Оператор and

Предположим, мы написали программу для учеников от двенадцати лет, которые учатся по крайней мере в 7 классе. Доступ к ней тем, кто младше, надо запретить. Следующий код решает поставленную задачу:

```
age = int(input('Сколько вам лет?: '))
grade = int(input('В каком классе вы учитесь?: '))
if age >= 12 and grade >= 7:
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

Мы объединили два условия при помощи оператора `and`. Он означает, что в этом ветвлении блок кода выполняется только при выполнении **обоих условий одновременно!**

Оператор `and` может объединять произвольное количество условий:

```
age = int(input('Сколько вам лет?: '))
grade = int(input('В каком классе вы учитесь?: '))
city = input('В каком городе вы живете?: ')
if age >= 12 and grade >= 7 and city == 'Москва':
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

Это таблица истинности для оператора `and`. В ней перечислены выражения, соединённые оператором `and`, показаны все возможные комбинации истинности и ложности и приведены результирующие значения выражений.

a	b	a and b
False	False	False
False	True	False
True	False	False
True	True	True

Как показывает таблица, чтобы значение выражения с оператором `and` было истинным, должны быть истинными **оба (все)** объединенные им условия.

Оператор or

Оператор `or` также применяется для объединения условий. Однако, в отличие от `and`, для выполнения блока кода достаточно выполнения **хотя бы одного из условий**.

```
city = input('В каком городе вы живете?: ')
if city == 'Москва' or city == 'Санкт-Петербург' or city == 'Екатеринбург':
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

Доступ будет разрешен в случае, если хотя бы одно из условий выполнится.

Это таблица истинности для оператора `or`. В ней перечислены выражения, соединённые оператором `or`, показаны все возможные комбинации истинности и ложности и приведены результирующие значения выражений.

a	b	a or b
False	False	False
False	True	True
True	False	True
True	True	True

Для того, чтобы выражение `or` было истинным, требуется, чтобы **хотя бы одно** условие оператора `or` было истинным. При этом не имеет значения, истинным или ложным является второе выражение.



Логическое выражение `X and Y` истинно, если **оба** значения X и Y истинны.

Логическое выражение `X or Y` истинно, если **хотя бы одно** из значений X и Y истинно.

Мы можем использовать оба логических оператора одновременно:

```
age = int(input('Сколько вам лет?: '))
grade = int(input('В каком классе вы учитесь?: '))
city = input('В каком городе вы живете?: ')
if age >= 12 and grade >= 7 and (city == 'Москва' or city == 'Санкт-Петербург'):
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

Такой код проверяет, что возраст учеников от двенадцати лет и учатся они по крайней мере в 7 классе и живут в Москве или Санкт-Петербурге.

Оператор not

Оператор `not` позволяет инвертировать (т.е. заменить на противоположный) результат логического выражения. Например, следующий код:

```
age = int(input('Сколько вам лет?: '))
if not (age < 12):
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

полностью эквивалентен коду:

```
age = int(input('Сколько вам лет?: '))
if age >= 12:
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

В первом примере мы поместили выражение `age < 12` в скобки для того, чтобы было чётко видно, что мы применяем оператор `not` к значению выражения `age < 12`, а не только к переменной `age`.

Таблица истинности для оператора `not`:

a	not a
False	True
True	False

Приоритеты логических операторов

Логические операторы, подобно арифметическим операторам (`+` , `-` , `*` , `/`), имеют приоритет выполнения. Приоритет выполнения следующий:

- в первую очередь выполняется логическое отрицание `not` ;
- далее выполняется логическое умножение `and` ;
- далее выполняется логическое сложение `or` .

Для **явного указания порядка** выполнения условных операторов **используют скобки**.

Примечания

Примечание 1. Частой ошибкой у начинающих программистов является путаница логических операторов `and` и `or` . Рассмотрим два условия:

```
if x > 1 and x < 100:  
if x > 1 or x < 100:
```

Верным является только первое условие. В нём проверяется, что число x находится в диапазоне от 1 до 100, другими словами, $x \in (1; 100)$. Второе условие проверяет, что число x или больше 1, или меньше 100. Однако такому условию удовлетворяет любое число!

Примечание 2. Другую частую ошибку видим в следующем примере кода:

```
if age >= 7 and <= 9:
```

Запуск такого кода приведет к появлению ошибки во время выполнения программы. Необходимо явно записывать условия:

```
if age >= 7 and age <= 9:
```

Примечание 3. Не забывайте, что в Python есть удобный способ для проверки принадлежности диапазону. Например, следующий код:

```
if age >= 7 and age <= 9:
```

полностью эквивалентен коду:

```
if 7 <= age <= 9:
```

Последний код предпочтительнее.

Примечание 4. Оба оператора, `and` и `or`, вычисляются по **укороченной схеме**.

Вот как это работает с оператором `and`. Если условие слева от оператора `and` является ложным, то условие справа от него не проверяется, так как результат выражения будет гарантированно ложным и проверка оставшегося условия — пустая трата процессорного времени.

Например, в таком выражении:

```
5 > 100 and 10 > 0
```

вычисляется только выражение `5 > 100`. Оно ложно (потому что 5 не может быть больше 100). При операторе `and` оба выражения должны быть правдивы, чтобы результат был `True`. Но у нас уже одно не правдиво, значит, результат и так будет `False`. Поэтому нам и не надо вычислять второе выражение, оно всё равно не повлияет на результат.

Аналогично работает оператор `or`. Если условие слева от оператора `or` истинное, то условие справа от него не проверяется. Действительно, результат будет гарантированно истинным и проверка оставшегося условия станет пустой тратой процессорного времени.

Например, в таком выражении:

```
10 > 0 or 5 > 100
```

вычисляется только выражение `10 > 0`. Оно правдиво, значит результат тоже правдив, т.к. нам достаточно одного правдивого выражения при операторе `or`.

Решение задач

Задача 1. Напишите программу, которая определяет, является ли заданное натуральное число трёхзначным.

Решение. Программа, решающая поставленную задачу, может иметь следующий вид:

```
num = int(input())
if 100 <= num <= 999:      # num >= 100 and num <= 999
    print('Число является трёхзначным')
else:
    print('Число не является трёхзначным')
```

Задача 2. Напишите программу, которая проверяет, что все три цифры натурального трёхзначного числа различны.

Решение. Программа, решающая поставленную задачу, может иметь следующий вид:

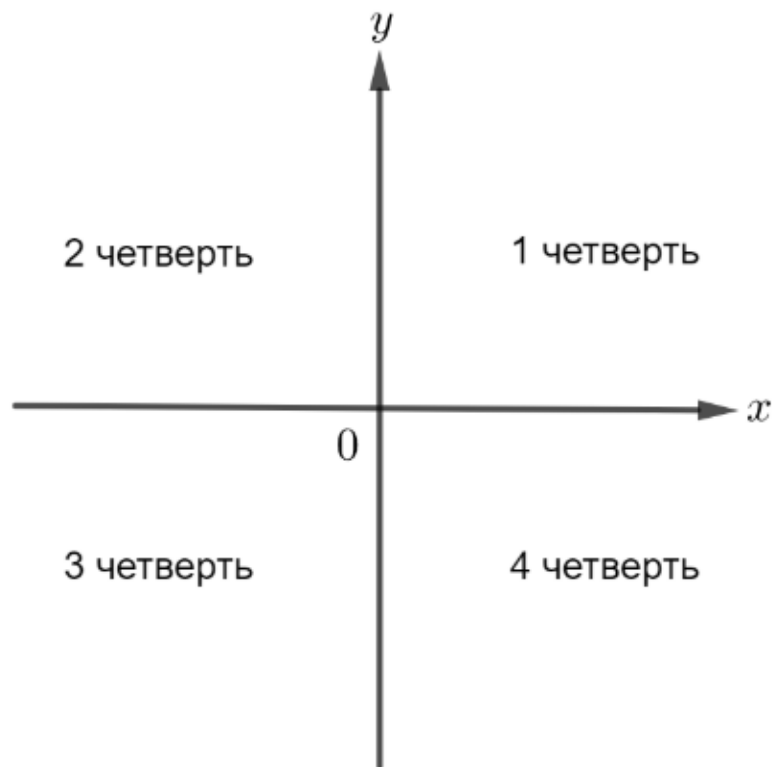
```
num = int(input())
d3 = num % 10
d2 = num % 100 // 10
d1 = num // 100
if d3 != d2 and d3 != d1 and d2 != d1:
    print('Цифры различны')
else:
    print('Цифры не различны')
```

Задача 3. Напишите программу, которая по координатам точки, не лежащей на осях координат, определяет номер координатной четверти, в которой она находится.

Решение. Программа, решающая поставленную задачу, может иметь следующий вид:

```
x = int(input())
y = int(input())

if x > 0 and y > 0:
    print('1 четверть')
if x < 0 and y > 0:
    print('2 четверть')
if x < 0 and y < 0:
    print('3 четверть')
if x > 0 and y < 0:
    print('4 четверть')
```



Примечание. Обратите внимание: никакие два из четырёх условий не могут быть истинными одновременно.

Тема урока: условный оператор

Вложенный оператор

Внутри условного оператора можно использовать любые инструкции языка Python, в том числе и условный оператор. Получаем вложенное ветвление: после одной развилки в ходе исполнения программы появляется другая развилка. При этом вложенные блоки имеют больший размер отступа (+4 пробела для каждого следующего уровня).

```
if условие1:
    блок кода
else:
    if условие2:
        блок кода
    else:
        if условие3:
            блок кода
        ...
```

В предыдущем уроке мы разбирали задачу об определении координатной четверти точки. Программу можно переписать с использованием вложенного оператора:

```
x = int(input())
y = int(input())

if x > 0:
    if y > 0:
        print('Первая четверть')
    else:
        print('Четвертая четверть')
else:
    if y > 0:
        print('Вторая четверть')
    else:
        print('Третья четверть')
```

В данном случае уровень вложенности равен двум, так что программа одинаково хорошо читается как с помощью использования логического оператора `and`, так и с помощью вложенного оператора.

Рассмотрим программу, которая переводит стобалльную оценку в пятибалльную. Для ее реализации нужно воспользоваться вложенным условным оператором:

```
grade = int(input('Введите вашу отметку по 100-балльной системе: '))

if grade >= 90:
    print(5)
else:
    if grade >= 80:
        print(4)
    else:
        if grade >= 70:
            print(3)
        else:
            if grade >= 60:
                print(2)
            else:
                print(1)
```

В этом примере уровень вложенности настолько глубок, что код становится трудно понять.

Выбор из нескольких альтернатив – это обычное дело, здесь имеет смысл избегать глубокого вложения. Для этого в Python есть **каскадный условный оператор**.



Мы не могли написать 5 независимых `if`-ов, поскольку в таком случае было бы напечатано сразу несколько значений пятибалльной оценки.

Каскадный условный оператор

Если требуется проверить несколько условий, в языке Python используется **каскадный условный оператор**.

Синтаксис каскадного условного оператора имеет следующий вид:

```
if условие1:  
    блок кода  
elif условие2:  
    блок кода  
...  
else:  
    блок кода
```



При исполнении такого условного оператора сначала проверяется условие1. Если оно является истинным, то выполняется блок кода, который следует сразу после него, вплоть до выражения `elif`. Остальная часть конструкции игнорируется. Однако если условие1 является ложным, то программа перескакивает непосредственно к следующему выражению `elif` и проверяет условие2. Если оно истинное, то выполняется блок кода, который следует сразу после него, вплоть до следующего выражения `elif`. И остальная часть условного оператора тогда игнорируется. Этот процесс продолжается до тех пор, пока не будет найдено условие, которое является истинным, либо пока больше не останется выражений `elif`. Если ни одно условие не является истинным, то выполняется блок кода после выражения `else`.

Приведенный ниже фрагмент кода является примером каскадного условного оператора `if-elif-else`. Этот фрагмент кода работает так же, как предыдущий код, использующий вложенный условный оператор.

```
grade = int(input('Введите вашу отметку: '))

if grade >= 90:
    print(5)
elif grade >= 80:
    print(4)
elif grade >= 70:
    print(3)
elif grade >= 60:
    print(2)
else:
    print(1)
```

Обратите внимание на выравнивание и выделение отступом, которые применены в инструкции `if-elif-else`: выражения `if`, `elif` и `else` выровнены и исполняемые по условию блоки выделены отступом.

Инструкция `if-elif-else` не является обязательной, потому что ее логика может быть запрограммирована вложенными инструкциями `if-else`. Однако длинная серия вложенных инструкций `if-else` имеет два характерных недостатка:

- программный код может стать сложным и трудным для восприятия;
- из-за необходимого выделения отступом продолжительная серия вложенных инструкций `if-else` может стать слишком длинной, чтобы целиком уместиться на экране монитора без горизонтальной прокрутки.

Логика инструкции `if-elif-else` обычно прослеживается легче, чем длинная серия вложенных инструкций `if-else`. И поскольку в инструкции `if-elif-else` все выражения выровнены, длина строк в данной инструкции, как правило, короче.



Запомни. Заключительный блок `else` в операторе `if-elif-else` является необязательным:

```
traffic_light_signal = input('Введите сигнал светофора: ')

if traffic_light_signal == 'красный':
    print('Стой!')
elif traffic_light_signal == 'желтый':
    print('Приготовься...')
elif traffic_light_signal == 'зеленый':
    print('Иди!')
```

В данном случае если будет введен некорректный сигнал светофора, программа ничего не выведет. Каждое условие будет проверено поочередно, и программа завершится без ошибок.

Конструкция match-case в Python

Начиная с версии 3.10 в языке Python наконец-то появилась конструкция switch-case, которая называется match-case.

С помощью выражения match-case можно избавиться от довольно громоздких цепочек if-elif-else, например:

```
http_status = 400
if http_status == 400:
    print("Bad Request")
elif http_status == 403:
    print("Forbidden")
elif http_status == 404:
    print("Not Found")
else:
    print("Other")
```

Вместо этого можно использовать компактное выражение match-case:

```
http_status = 400
match http_status:
    case 400:
        print("Bad Request")
    case 403:
        print("Forbidden")
    case 404:
        print("Not Found")
    case _:
        print("Other")
```

Во многих случаях последний вариант гораздо лучше. Он делает код более читаемым и менее повторяемым.

match-case в Python

Общая структура match-case в Python имеет следующий синтаксис:

```
match element:
    case pattern1:
        # statements
    case pattern2:
        # statements
    case pattern3:
        # statements
```

В этой конструкции кода:

- **match element** означает «сопоставьте элемент со следующими шаблонами»
- затем каждое выражение **case pattern** сравнивает элемент с указанным шаблоном (это может быть, например, строка или число)
- если шаблон соответствует элементу, выполняется соответствующий блок кода, после этого оператор match-case заканчивает свою работу.

В общем, все то же самое, что и с применением оператора if-elif-else, но с меньшим количеством повторений.

Задачи:

Начало столетия

Напишите программу, которая определяет, оканчивается ли год с данным номером на два нуля. Если год оканчивается, то выведите «YES» (без кавычек), иначе выведите «NO» (без кавычек).

Формат входных данных

На вход программе подаётся натуральное число.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

Шахматная доска

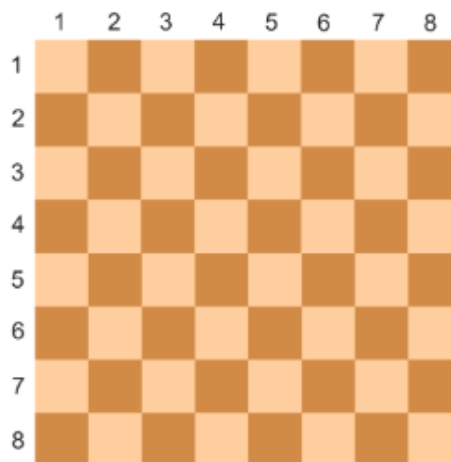
Заданы две клетки шахматной доски. Напишите программу, которая определяет, имеют ли указанные клетки один цвет или нет. Если они покрашены в один цвет, то выведите слово «YES» (без кавычек), а если в разные цвета, то «NO» (без кавычек).

Формат входных данных

На вход программе подаются четыре числа от 1 до 8 каждое, задающие номер столбца и номер строки сначала для первой клетки, потом для второй клетки.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.



Girls only 🧒

Футбольная команда набирает девочек от 10 до 15 лет включительно. Напишите программу, которая запрашивает возраст и пол претендента, используя для обозначения пола буквы «m» (от male – мужчина) и «f» (от female – женщина), и определяет, подходит ли претендент для вступления в команду или нет. Если претендент подходит, то выведите «YES» (без кавычек), иначе выведите «NO» (без кавычек).

Формат входных данных

На вход программе подаётся натуральное число – возраст претендента и буква обозначающая пол m (мужчина) или f (женщина).

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

Римские цифры

Напишите программу, которая считывает целое число и выводит соответствующую ему римскую цифру. Если число находится вне диапазона [1; 10], то программа должна вывести текст «ошибка» (без кавычек).

В таблице приведены римские цифры для чисел от 1 до 10:

Число	Римская цифра
1	I
2	II
3	III
4	IV
5	V
6	VI
7	VII
8	VIII
9	IX
10	X

Формат входных данных

На вход программе подаётся целое число.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

YES or NO – вот в чём вопрос ?

Напишите программу, которая принимает на вход число и в зависимости от условий выводит текст «YES» (без кавычек) либо «NO» (без кавычек).

Условия:

- если число нечётное, то вывести «YES»;
- если число чётное в диапазоне от 2 до 5 (включительно), то вывести «NO»;
- если число чётное в диапазоне от 6 до 20 (включительно), то вывести «YES»;
- если число чётное и больше 20, то вывести «NO».

Формат входных данных

На вход программе подаётся натуральное число.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

Ход слона ♗ 🌶️

Даны две **различные** клетки шахматной доски. Напишите программу, которая определяет, может ли слон попасть с первой клетки на вторую одним ходом. Программа получает на вход четыре числа от 1 до 8 каждое, задающие номер столбца и номер строки сначала для первой клетки, потом для второй клетки. Программа должна вывести «YES», если из первой клетки ходом слона можно попасть во вторую, или «NO» в противном случае.

Формат входных данных

На вход программе подаются четыре числа от 1 до 8.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

Примечание. Шахматный слон ходит по диагоналям.



Ход коня ♞

Даны две **различные** клетки шахматной доски. Напишите программу, которая определяет, может ли конь попасть с первой клетки на вторую одним ходом. Программа получает на вход четыре числа от 1 до 8 каждое, задающие номер столбца и номер строки сначала для первой клетки, потом для второй клетки. Программа должна вывести «YES», если из первой клетки ходом коня можно попасть во вторую, или «NO» в противном случае.

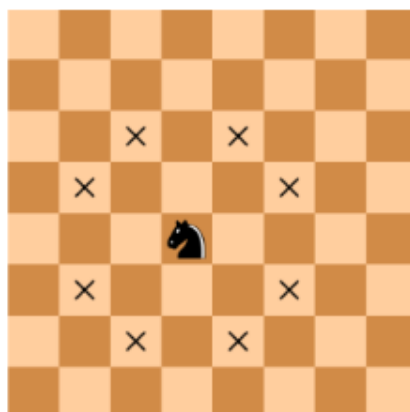
Формат входных данных

На вход программе подаются четыре числа от 1 до 8.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

Примечание. Шахматный конь ходит буквой «Г».



Ход ферзя ♔

Даны две **различные** клетки шахматной доски. Напишите программу, которая определяет, может ли ферзь попасть с первой клетки на вторую одним ходом. Программа получает на вход четыре числа от 1 до 8 каждое, задающие номер столбца и номер строки сначала для первой клетки, потом для второй клетки. Программа должна вывести «YES», если из первой клетки ходом ферзя можно попасть во вторую, или «NO» в противном случае.

Формат входных данных

На вход программе подаются четыре числа от 1 до 8.

Формат выходных данных

Программа должна вывести текст в соответствии с условием задачи.

Примечание. Шахматный ферзь ходит по диагонали, горизонтали или вертикали.

